



BME513 - Inteligencia Artificial en Salud

Backpropagation

Alejandro Veloz

Gradients and initialization

How to calculate the gradients efficiently?

Largest models have $\sim 10^{12}$ parameters.

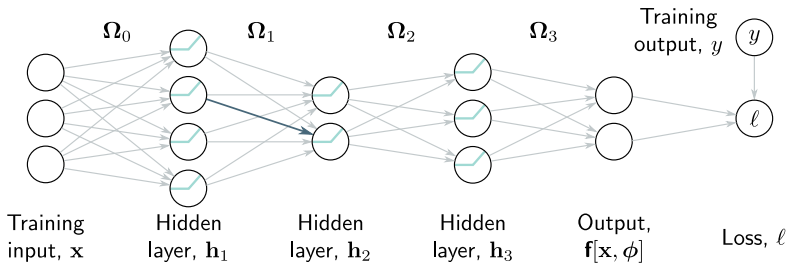
The gradient is computed for every parameter at every iteration of the training algorithm.

How to initialize the parameters?

Initial losses and their gradients can be extremely large or small. This impedes the training process.

Forward pass

Consider a network $f[\mathbf{x}, \phi]$:



$$\mathbf{h}_1 = \mathbf{a} [\beta_0 + \Omega_0 \mathbf{x}]$$

$$\mathbf{h}_2 = \mathbf{a} [\beta_1 + \Omega_1 \mathbf{h}_1]$$

$$\mathbf{h}_3 = \mathbf{a} [\beta_2 + \Omega_2 \mathbf{h}_2]$$

$$\mathbf{f}[\mathbf{x}, \phi] = \beta_3 + \Omega_3 \mathbf{h}_3$$

$\mathbf{a}[\bullet]$ applies the activation function elementwise.

Model parameters:

$$\phi = \{\beta_0, \Omega_0, \beta_1, \Omega_1, \beta_2, \Omega_2, \beta_3, \Omega_3\}$$

Total loss (sum of individual losses):

$$L[\phi] = \sum_{i=1}^I \ell_i$$

Individual loss term ℓ_i $\rightarrow$ **Negative log-likelihood** of the ground truth label y_i given the model prediction $f[\mathbf{x}_i, \phi]$ for training input $\mathbf{x}_i$.

Stochastic gradient descent (SGD) updates the parameters as:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i[\phi_t]}{\partial \phi}$$

α is the learning rate; $\mathcal{B}_t$ contains the batch indices at iteration t .

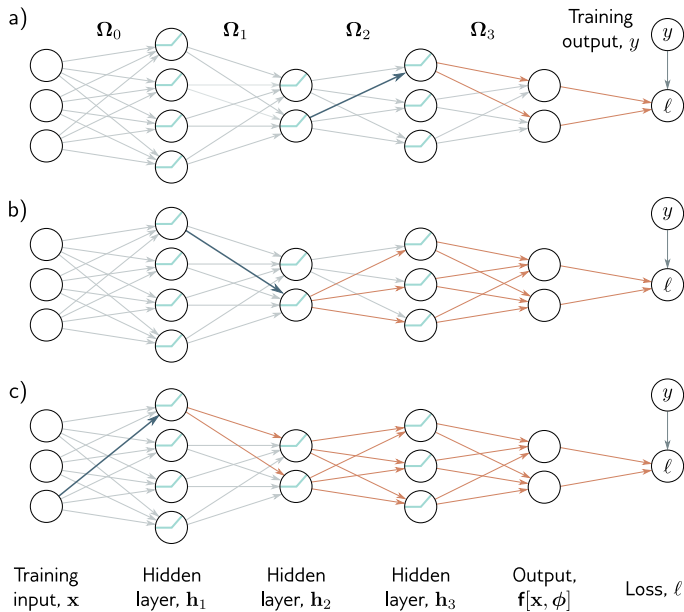
To compute the SGD update, we need:

$$\frac{\partial \ell_i}{\partial \beta_k} \quad \text{and} \quad \frac{\partial \ell_i}{\partial \Omega_k}$$

for the parameters $\{\beta_k, \Omega_k\}$ at every layer $k \in \{0, 1, \dots, K\}$ and for each index i in the batch.

Backpropagation algorithm computes these derivatives efficiently.

Forward pass



Toy example

Consider a model $f[x, \phi]$ with parameters

$$\phi = \{\beta_0, \omega_0, \beta_1, \omega_1, \beta_2, \omega_2, \beta_3, \omega_3\}:$$

$$f[x, \phi] = \beta_3 + \omega_3 \cos [\beta_2 + \omega_2 \exp [\beta_1 + \omega_1 \sin [\beta_0 + \omega_0 x]]]$$

and a least squares loss function $L[\phi] = \sum_i \ell_i$ with individual terms:

$$\ell_i = (f[x_i, \phi] - y_i)^2$$

[neural network with one input, one output, one hidden unit at each layer, and activation functions $\sin[\bullet]$, $\exp[\bullet]$, and $\cos[\bullet]$ between each layer]

We aim to compute the derivatives:

$$\frac{\partial \ell_i}{\partial \beta_0}, \frac{\partial \ell_i}{\partial \omega_0}, \frac{\partial \ell_i}{\partial \beta_1}, \frac{\partial \ell_i}{\partial \omega_1}, \frac{\partial \ell_i}{\partial \beta_2}, \frac{\partial \ell_i}{\partial \omega_2}, \frac{\partial \ell_i}{\partial \beta_3}, \text{ and } \frac{\partial \ell_i}{\partial \omega_3}.$$

For example:

$$\begin{aligned} \frac{\partial \ell_i}{\partial \omega_0} = & -2 (\beta_3 + \omega_3 \cos [\beta_2 + \omega_2 \exp [\beta_1 + \omega_1 \sin [\beta_0 + \omega_0 x_i]]] - y_i) \\ & \omega_1 \omega_2 \omega_3 x_i \cos [\beta_0 + \omega_0 x_i] \exp [\beta_1 + \omega_1 \sin [\beta_0 + \omega_0 x_i]] \\ & \sin [\beta_2 + \omega_2 \exp [\beta_1 + \omega_1 \sin [\beta_0 + \omega_0 x_i]]] . \end{aligned}$$

[Such expressions do not exploit the inherent redundancy; note that the three exponential terms are the same]

Backpropagation algorithm

Efficient method for computing all of these derivatives at once.

It consists of:

- (i) **A forward pass**, in which we compute and store a series of intermediate values and the network output.
- (ii) **A backward pass**, in which we calculate the derivatives of each parameter, starting at the end of the network, and reusing previous calculations as we move toward the start.

Forward pass

We treat the computation of the loss as a series of calculations:

$$f_0 = \beta_0 + \omega_0 x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + \omega_1 h_1$$

$$h_2 = \exp[f_1]$$

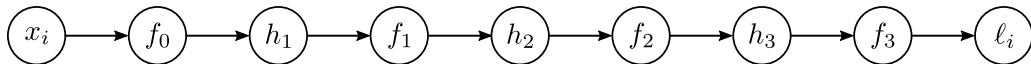
$$f_2 = \beta_2 + \omega_2 h_2$$

$$h_3 = \cos[f_2]$$

$$f_3 = \beta_3 + \omega_3 h_3$$

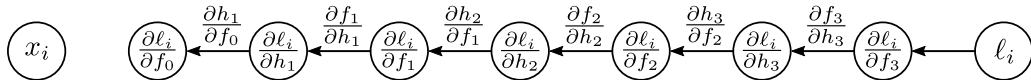
$$\ell_i = (f_3 - y_i)^2.$$

We compute and store the values of the intermediate variables f_k and h_k .



Backward pass #1

Compute the derivatives of ℓ_i with respect to these intermediate variables, but in reverse order.



The first of these derivatives is:

$$\frac{\partial \ell_i}{\partial f_3} = 2(f_3 - y_i)$$

The next derivative can be calculated using the chain rule:

$$\frac{\partial \ell_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3}$$

We continue in this way, computing the derivatives of the output with respect to these intermediate quantities:

$$\frac{\partial \ell_i}{\partial f_2} = \frac{\partial h_3}{\partial f_2} \left(\frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial h_2} = \frac{\partial f_2}{\partial h_2} \left(\frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial f_1} = \frac{\partial h_2}{\partial f_1} \left(\frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial h_1} = \frac{\partial f_1}{\partial h_1} \left(\frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

$$\frac{\partial \ell_i}{\partial f_0} = \frac{\partial h_1}{\partial f_0} \left(\frac{\partial f_1}{\partial h_1} \frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)$$

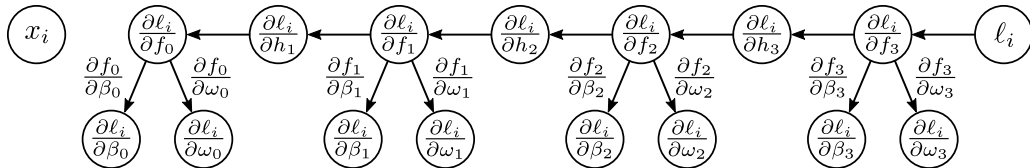
Backward pass #2

Finally, we consider how the loss ℓ_i changes when we change the parameters $\{\beta_k\}$ and $\{\omega_k\}$.

Once more, we apply the chain rule:

$$\frac{\partial \ell_i}{\partial \beta_k} = \frac{\partial f_k}{\partial \beta_k} \frac{\partial \ell_i}{\partial f_k}$$

$$\frac{\partial \ell_i}{\partial \omega_k} = \frac{\partial f_k}{\partial \omega_k} \frac{\partial \ell_i}{\partial f_k}$$



When $k > 0$, $f_k = \beta_k + \omega_k h_k$, therefore:

$$\frac{\partial f_k}{\partial \beta_k} = 1 \quad \text{and} \quad \frac{\partial f_k}{\partial \omega_k} = h_k$$

The final derivatives from the term $f_0 = \beta_0 + \omega_0 x_i$ are:

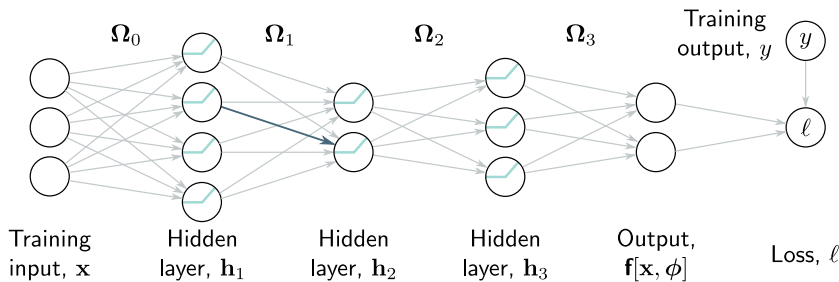
$$\frac{\partial f_0}{\partial \beta_0} = 1 \quad \text{and} \quad \frac{\partial f_0}{\partial \omega_0} = x_i$$

The effect of a change in the weight ω_k is proportional to the value of the source variable h_k (which was stored in the forward pass).

Backpropagation is both simpler and more efficient than computing the derivatives individually.

Backpropagation algorithm

We now repeat this process for a three-layer network.



The intermediate variables $\mathbf{f}_k$, $\mathbf{h}_k$, and the biases β_k are vectors, the weights Ω_k are matrices.

We use ReLU as activation functions.

Forward pass

We write the network as a series of sequential calculations:

$$\mathbf{f}_0 = \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i$$

$$\mathbf{h}_1 = \mathbf{a}[\mathbf{f}_0]$$

$$\mathbf{f}_1 = \boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{h}_1$$

$$\mathbf{h}_2 = \mathbf{a}[\mathbf{f}_1]$$

$$\mathbf{f}_2 = \boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{h}_2$$

$$\mathbf{h}_3 = \mathbf{a}[\mathbf{f}_2]$$

$$\mathbf{f}_3 = \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3$$

$$\ell_i = l[\mathbf{f}_3, y_i]$$

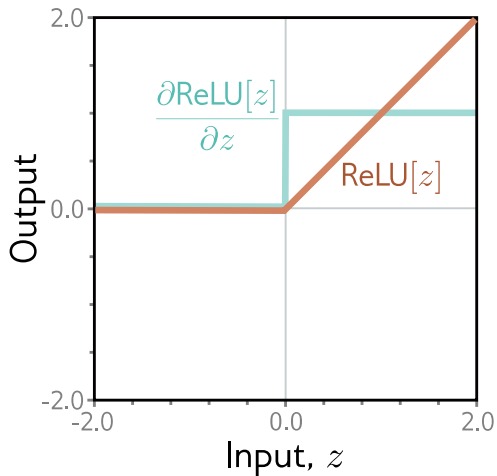
$\mathbf{f}_{k-1} \rightarrow$ pre-activations at the k^{th} hidden layer (i.e., the values before the ReLU function $\mathbf{a}[\bullet]$).

$\mathbf{h}_k \rightarrow$ activations at the k^{th} hidden layer (i.e., after the ReLU).

$\ell[\mathbf{f}_3, y_i] \rightarrow$ loss function (e.g., least squares or binary cross-entropy loss).

We store all the intermediate quantities.

ReLU and its derivative



Backward pass #1

How the loss changes when the pre-activations f_0, f_1, f_2 change?

Derivative of the loss ℓ_i with respect to f_2 :

$$\frac{\partial \ell_i}{\partial f_2} = \underbrace{\frac{\partial \mathbf{h}_3}{\partial f_2}}_{D_3 \times D_3} \underbrace{\frac{\partial f_3}{\partial \mathbf{h}_3}}_{D_3 \times D_f} \underbrace{\frac{\partial \ell_i}{\partial f_3}}_{D_f \times 1}$$

where D_3 is the number of hidden units in the third layer, and D_f is the dimensionality of the model output f_3 .

Loss derivative w.r.t. f_1 and f_0 :

$$\frac{\partial \ell_i}{\partial f_1} = \frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \underbrace{\left(\frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)}_{\frac{\partial \ell_i}{\partial f_2}}$$

$$\frac{\partial \ell_i}{\partial f_0} = \frac{\partial h_1}{\partial f_0} \frac{\partial f_1}{\partial h_1} \underbrace{\left(\frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial \ell_i}{\partial f_3} \right)}_{\frac{\partial \ell_i}{\partial f_1}}$$

Working backward through $\frac{\partial \ell_i}{\partial \mathbf{f}_2} = \frac{\partial \mathbf{h}_3}{\partial \mathbf{f}_2} \frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} \frac{\partial \ell_i}{\partial \mathbf{f}_3}$, we have:

- $\partial \ell_i / \partial \mathbf{f}_3 \rightarrow$ depend on the loss function.
- $\frac{\partial \mathbf{f}_3}{\partial \mathbf{h}_3} = \frac{\partial}{\partial \mathbf{h}_3} (\beta_3 + \mathbf{\Omega}_3 \mathbf{h}_3) = \mathbf{\Omega}_3^\top$
- $\partial \mathbf{h}_3 / \partial \mathbf{f}_2 \rightarrow$ depend on the activation function.

$$\partial \mathbf{h}_3 / \partial \mathbf{f}_2$$

- It will be a diagonal matrix since each activation only depends on the corresponding pre-activation.
- For ReLU functions, the diagonal terms are zero everywhere f_2 is less than zero and one otherwise.
- For efficiency: Rather than multiply by this matrix, we extract the diagonal terms as a vector $\mathbb{I}[f_2 > 0]$ and pointwise multiply.

The terms on the right-hand side of $\frac{\partial \ell_i}{\partial \mathbf{f}_1}$ and $\frac{\partial \ell_i}{\partial \mathbf{f}_0}$ have similar forms.

As we progress back through the network, we alternately:

- (i) multiply by the transpose of the weight matrices $\Omega_k^\top$,
and
- (ii) threshold based on the inputs $\mathbf{f}_{k-1}$ to the hidden layer.
These inputs were stored during the forward pass.

Backward pass #2

To calculate the derivatives of the loss with respect to the biases β_k , we again use the chain rule:

$$\begin{aligned}\frac{\partial \ell_i}{\partial \beta_k} &= \frac{\partial \mathbf{f}_k}{\partial \beta_k} \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ &= \frac{\partial}{\partial \beta_k} (\beta_k + \mathbf{\Omega}_k \mathbf{h}_k) \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ &= \frac{\partial \ell_i}{\partial \mathbf{f}_k}\end{aligned}$$

Similarly, the derivative for the weights matrix $\mathbf{\Omega}_k$, is given by:

$$\begin{aligned}\frac{\partial \ell_i}{\partial \mathbf{\Omega}_k} &= \frac{\partial \mathbf{f}_k}{\partial \mathbf{\Omega}_k} \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ &= \frac{\partial}{\partial \mathbf{\Omega}_k} (\beta_k + \mathbf{\Omega}_k \mathbf{h}_k) \frac{\partial \ell_i}{\partial \mathbf{f}_k} \\ &= \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^\top \rightarrow \text{matrix of the same size as } \mathbf{\Omega}_k\end{aligned}$$

Backpropagation algorithm summary

Consider a deep neural network $f[\mathbf{x}_i, \phi]$ that:

- takes input $\mathbf{x}_i$,
- has K hidden layers with ReLU activations, and
- individual loss term $\ell_i = l[f[\mathbf{x}_i, \phi], \mathbf{y}_i]$.

The goal of backpropagation is to compute $\partial \ell_i / \partial \beta_k$ and $\partial \ell_i / \partial \Omega_k$.

Forward pass. Compute and store the following quantities:

$$\begin{aligned} \mathbf{f}_0 &= \boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i \\ \mathbf{h}_k &= \mathbf{a}[\mathbf{f}_{k-1}] & k \in \{1, 2, \dots, K\} \\ \mathbf{f}_k &= \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{h}_k & k \in \{1, 2, \dots, K\} \end{aligned}$$

Backward pass. Start with the derivative $\partial \ell_i / \partial \mathbf{f}_K$ and work backward through the network:

$$\frac{\partial \ell_i}{\partial \boldsymbol{\beta}_k} = \frac{\partial \ell_i}{\partial \mathbf{f}_k}, \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial \ell_i}{\partial \boldsymbol{\Omega}_k} = \frac{\partial \ell_i}{\partial \mathbf{f}_k} \mathbf{h}_k^\top, \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial \ell_i}{\partial \mathbf{f}_{k-1}} = \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\boldsymbol{\Omega}_k^\top \frac{\partial \ell_i}{\partial \mathbf{f}_k} \right) \quad k \in \{K, K-1, \dots, 1\}$$

where $\odot$ denotes pointwise multiplication, and $\mathbb{I}[\mathbf{f}_{k-1} > 0]$ is a vector containing ones where $\mathbf{f}_{k-1}$ is greater than zero and zeros elsewhere.

Finally, compute the derivatives w.r.t. the first set of biases and weights:

$$\frac{\partial \ell_i}{\partial \beta_0} = \frac{\partial \ell_i}{\partial \mathbf{f}_0}$$
$$\frac{\partial \ell_i}{\partial \Omega_0} = \frac{\partial \ell_i}{\partial \mathbf{f}_0} \mathbf{x}_i^\top$$

The derivatives are calculated for every training example in the batch and summed to obtain the gradient for the SGD update.

Backpropagation algorithm **is extremely efficient**.

However, it **is not memory efficient**; the intermediate values in the forward pass must all be stored, and this can limit the size of the models that can be trained.

Algorithmic differentiation

Modern deep learning frameworks such as PyTorch and TensorFlow calculate the derivatives automatically (algorithmic differentiation).

Each functional component (linear transform, ReLU activation, loss function) in the framework knows how to compute its own derivative.

Computations such as matrix multiplication are naturally amenable to parallelization.

Since the training algorithm now processes the entire batch in parallel, the input becomes a multi-dimensional tensor.

Arbitrary computational graphs

Backpropagation (BP) is not limited to sequential models, even though standard deep networks typically calculate intermediate quantities $(f_0, \mathbf{h}_1, \dots, f_k)$ in a linear order.

BP naturally extends to branching structures, where hidden layer values are processed through multiple sub-networks before being recombined.

The fundamental requirement for backpropagation to function is that the computational graph must be acyclic.

Parameter initialization

During the forward pass, each set of pre-activations $\mathbf{f}_k$ is computed as:

$$\begin{aligned}\mathbf{f}_k &= \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{h}_k \\ &= \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{a}[\mathbf{f}_{k-1}]\end{aligned}$$

$\mathbf{a}[\bullet] \rightarrow$ ReLU functions

$\boldsymbol{\Omega}_k \rightarrow$ weights

$\boldsymbol{\beta}_k \rightarrow$ biases

Imagine that we **initialize** all the **biases to zero** and the elements of $\boldsymbol{\Omega}_k$ **according to a normal distribution** with mean zero and variance σ^2 .

Parameter initialization

Two scenarios:

1. If the variance σ^2 is very small (e.g., 10^{-5}), then each element of $\beta_k + \Omega_k \mathbf{h}_k$ will be a weighted sum of $\mathbf{h}_k$ where the weights are very small; the result will likely have a smaller magnitude than the input.

In addition, the ReLU function clips values less than zero, so the range of $\mathbf{h}_k$ will be half that of $\mathbf{f}_{k-1}$.

Consequently, the magnitudes of the pre-activations at the hidden layers will get smaller and smaller as we progress through the network.

Parameter initialization

Two scenarios:

2. If the variance σ^2 is very large (e.g., 10^5), then each element of $\beta_k + \Omega_k \mathbf{h}_k$ will be a weighted sum of $\mathbf{h}_k$ where the weights are very large; the result is likely to have a much larger magnitude than the input.

The ReLU function halves the range of the inputs, but if σ^2 is large enough, the magnitudes of the pre-activations will still get larger as we progress through the network.

Parameter initialization

In these two situations, the values at the pre-activations can become so small or so large that they cannot be represented with finite precision floating point arithmetic.

Vanishing and exploding gradient problems

The same logic applies to the backward pass.

Each gradient update consists of multiplying by Ω^T .

If the values of Ω are not initialized sensibly, then the gradient magnitudes may decrease (**vanishing gradient problem**) or increase (**exploding gradient problem**) uncontrollably.

Parameters updates become vanishingly small or unstable.

Initialization for forward pass

Consider the computation between adjacent pre-activations f and f' with dimensions D_h and $D_{h'}$, respectively:

$$\mathbf{h} = \mathbf{a}[f]$$

$$f' = \beta + \Omega \mathbf{h}$$

Assume the pre-activations f_j in the input layer f have variance σ_f^2 .

Consider $\beta_i = 0$ and $\Omega_{ij} \sim \mathcal{N}(0, \sigma_\Omega^2)$.

Initialization for forward pass

The expectation (mean) $\mathbb{E}[f'_i]$ of the intermediate values f'_i is:

$$\begin{aligned}\mathbb{E}[f'_i] &= \mathbb{E}\left[\beta_i + \sum_{j=1}^{D_h} \Omega_{ij} h_j\right] = \mathbb{E}[\beta_i] + \sum_{j=1}^{D_h} \mathbb{E}[\Omega_{ij} h_j] \\ &= \mathbb{E}[\beta_i] + \sum_{j=1}^{D_h} \mathbb{E}[\Omega_{ij}] \mathbb{E}[h_j] \\ &= 0 + \sum_{j=1}^{D_h} 0 \mathbb{E}[h_j] = 0\end{aligned}$$

where D_h is the dimensionality of the input layer $\mathbf{h}$.

Using this result, we see that the variance $\sigma_{f'}^2$, of the pre-activations f'_i is:

$$\begin{aligned}\sigma_{f'_i}^2 &= \mathbb{E} [f_i'^2] - \mathbb{E} [f'_i]^2 = \mathbb{E} \left[\left(\beta_i + \sum_{j=1}^{D_h} \Omega_{ij} h_j \right)^2 \right] - 0 \\ &= \mathbb{E} \left[\left(\sum_{j=1}^{D_h} \Omega_{ij} h_j \right)^2 \right] \\ &= \sum_{j=1}^{D_h} \mathbb{E} [\Omega_{ij}^2] \mathbb{E} [h_j^2] \\ &= \sum_{j=1}^{D_h} \sigma_{\Omega}^2 \mathbb{E} [h_j^2] = \sigma_{\Omega}^2 \sum_{j=1}^{D_h} \mathbb{E} [h_j^2]\end{aligned}$$

we have used the identity $\sigma^2 = \mathbb{E} [(z - \mathbb{E}[z])^2] = \mathbb{E} [z^2] - \mathbb{E}[z]^2$.

Assuming that the distribution of pre-activations f_j at the previous layer is symmetric about zero, half of these pre-activations will be clipped by the ReLU function.

The second moment $\mathbb{E} [h_j^2]$ will be half the variance σ_f^2 of f_j :

$$\sigma_{f'_i}^2 = \sigma_{\Omega}^2 \sum_{j=1}^{D_h} \frac{\sigma_f^2}{2} = \frac{1}{2} D_h \sigma_{\Omega}^2 \sigma_f^2$$

This implies that if we want the variance $\sigma_{f'}^2$ of the subsequent pre-activations f' to be the same as the variance σ_f^2 of the original pre-activations f during the forward pass, we should set:

$$\sigma_{\Omega}^2 = \frac{2}{D_h} \quad [\text{He initialization}]$$

where D_h is the dimension of the original layer to which the weights were applied.

Initialization for backward pass

A similar argument establishes how the variance of the gradients $\partial \ell / \partial f_k$ changes during the backward pass.

During the backward pass, we multiply by $\Omega^\top$, so the equivalent expression becomes:

$$\sigma_\Omega^2 = \frac{2}{D_{h'}}$$

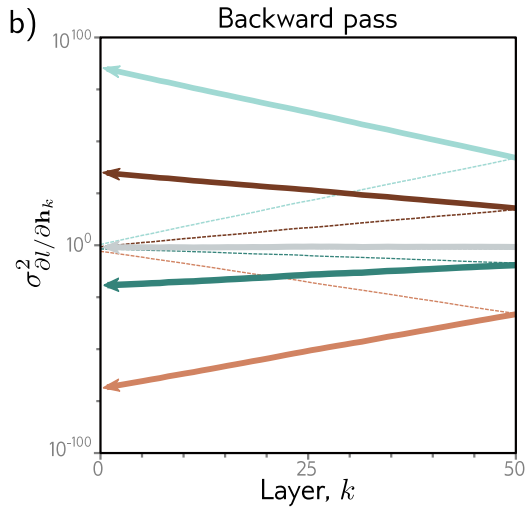
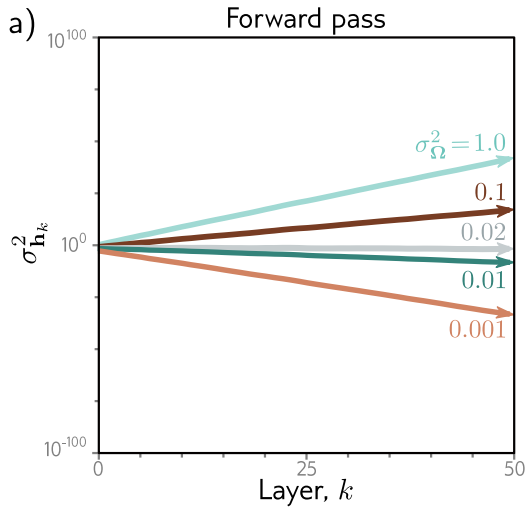
where $D_{h'}$ is the dimension of the layer that the weights feed into.

Initialization for both forward and backward pass

If the weight matrix Ω is not square (i.e. $D_h \neq D_{h'}$), then it is not possible to choose equivalent variances.

One possible compromise is to use the mean $(D_h + D_{h'}) / 2$ as a proxy for the number of terms, which gives:

$$\sigma_{\Omega}^2 = \frac{4}{D_h + D_{h'}}$$



Example training code

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.optim.lr_scheduler import StepLR
# define input size, hidden layer size, output size
D_i, D_k, D_o = 10, 40, 5
# create model with two hidden layers
model = nn.Sequential(
    nn.Linear(D_i, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_k),
    nn.ReLU(),
    nn.Linear(D_k, D_o))
```

```

# He initialization of weights
def weights_init(layer_in):
    if isinstance(layer_in, nn.Linear):
        nn.init.kaiming_normal_(layer_in.weight)
        layer_in.bias.data.fill_(0.0)
model.apply(weights_init)
# choose least squares loss function
criterion = nn.MSELoss()
# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.1, momentum=0.9)
# object that decreases learning rate by half every 10 epochs
scheduler = StepLR(optimizer, step_size=10, gamma=0.5)
# create 100 random data points and store in data loader class
x = torch.randn(100, D_i)
y = torch.randn(100, D_o)
data_loader = DataLoader(TensorDataset(x,y), batch_size=10, shuffle=True)

```

```
# loop over the dataset 100 times
for epoch in range(100):
    epoch_loss = 0.0
    # loop over batches
    for i, data in enumerate(data_loader):
        # retrieve inputs and labels for this batch
        x_batch, y_batch = data
        # zero the parameter gradients
        optimizer.zero_grad()
        # forward pass
        pred = model(x_batch)
        loss = criterion(pred, y_batch)
        # backward pass
        loss.backward()
        # SGD update
        optimizer.step()
        # update statistics
        epoch_loss += loss.item()
    # print error
    print(f'Epoch {epoch:5d}, loss {epoch_loss:.3f}')
    # tell scheduler to consider updating learning rate
    scheduler.step()
```